

SR UNIVERSITY

Campus Warangal

Program: II – B. Tech (CS&AI) **Department:** Computer Science and AI **Semester:** I

AI Assisted Coding – Assignment 11 **Date:** 21-09-2026

ASSIGNMENT 11 – NON-LINEAR DATA STRUCTURES

Name	Billa Vignesh	Reg. No.	2503A52335
-------------	---------------	-----------------	------------

Tasks 1–5	Tree • Graph • Heap • Trie • AI Decision Challenge
------------------	--

TASK 1 | TREE: HOSPITAL DEPARTMENT HIERARCHY

1. Question

A hospital has a hierarchical structure. Represent the following using a Tree:

Hospital

■ ■ ■ Cardiology

■ ■ ■ ■ ICU

■ ■ ■ ■ General Cardiology

■ ■ ■ Neurology

■ ■ ■ ■ Stroke Unit

■ ■ ■ ■ Neuro ICU

■ ■ ■ Orthopedics

Requirements:

1. Create tree nodes.

2. Add child departments.

3. Display the hierarchy.

4. Search for a department.

5. Count total departments.

6. Test at least three search cases.

2. Prompt

Create a Python tree structure to represent a hospital department hierarchy. Implement adding child nodes, displaying the hierarchy, searching for a department, and counting nodes. Explain how a tree is different from a list and provide test cases.

3. Code

```
"""Tree structure for a hospital department hierarchy."""
class DepartmentNode:
    """A hospital department and its child departments."""
    def __init__(self, name: str) -> None:
        if not name.strip():
            raise ValueError("Department name cannot be empty")
        self.name = name
        self.children: list[DepartmentNode] = []
        def add_child(self, child: "DepartmentNode | str") -> "DepartmentNode":
```

```

"""Add a department below this one and return the added node."""
if isinstance(child, str):
    child = DepartmentNode(child)
    self.children.append(child)
    return child
    def display(self, level: int = 0) -> str:
"""Display this department and its child departments."""
lines = [f'{' ' * level}- {self.name}']
for child in self.children:
    lines.append(child.display(level + 1))
return "\n".join(lines)
    def search(self, department_name: str) -> "DepartmentNode | None":
"""Search for a department by name."""
if self.name.casefold() == department_name.casefold():
    return self
for child in self.children:
    result = child.search(department_name)
if result is not None:
    return result
return None
    def count_nodes(self) -> int:
"""Count all departments in this tree."""
return 1 + sum(child.count_nodes() for child in self.children)
# Create hospital tree
hospital = DepartmentNode("Hospital")
# Cardiology
cardiology = hospital.add_child("Cardiology")
cardiology.add_child("ICU")
cardiology.add_child("General Cardiology")
# Neurology
neurology = hospital.add_child("Neurology")
neurology.add_child("Stroke Unit")
neurology.add_child("Neuro ICU")
# Orthopedics
hospital.add_child("Orthopedics")
# Display hierarchy
print("Hospital Department Hierarchy")
print("-----")
print(hospital.display())
# Search test cases
print("\nDepartment Search Results")
print("-----")
search_cases = ["ICU", "Neurology", "Emergency"]
for department in search_cases:
    result = hospital.search(department)

```

```

if result is not None:
    print(f"Search '{department}': Found")
else:
    print(f"Search '{department}': Not Found")

# Count total departments
print("\nTotal Departments")
print("-----")
print("Total departments:", hospital.count_nodes())

```

4. Output

Hospital Department Hierarchy

- Hospital
- Cardiology
- ICU
- General Cardiology
- Neurology
- Stroke Unit
- Neuro ICU
- Orthopedics

Department Search Results

Search 'ICU': Found

Search 'Neurology': Found

Search 'Emergency': Not Found

Total Departments

Total departments: 8

5. Observation

The hospital department hierarchy was successfully implemented using a Tree data structure in Python.

- The tree nodes were created successfully.
- Child departments were added under the correct parent departments.
- The complete hierarchy was displayed correctly.
- Three search cases were tested: ICU and Neurology were found, while Emergency was not found.
- The total number of nodes/departments, including the Hospital root, was 8.
- A Tree is more suitable than a simple list because it naturally represents parent-child relationships in a hierarchy.
- Search and counting operations in the implemented general tree have $O(n)$ worst-case time complexity.
- The program was executed and the output matched the expected results.

TASK 2 | GRAPH: HOSPITAL ROUTE FINDER

1. Question

Represent hospital locations as nodes and connections as edges. Display the adjacency list and find routes between locations using Breadth-First Search (BFS). Test at least three source-destination pairs.

2. Prompt

Create a Python graph for a hospital route finder. Add hospital locations as nodes and connections as edges. Display the adjacency list, find a route between locations using BFS, explain BFS step by step, and test at least three source-destination pairs.

3. Code

```
"""Task 2 - Hospital location graph with breadth-first route finding."""

from collections import deque
import unittest

class HospitalGraph:
    """An undirected graph of hospital locations and connecting corridors."""

    def __init__(self):
        self.locations = {}

    def add_node(self, location):
        """Add a location if it is not already in the graph."""
        self.locations.setdefault(location, set())

    def add_edge(self, first_location, second_location):
        """Connect two locations in both directions."""
        self.add_node(first_location)
        self.add_node(second_location)
        self.locations[first_location].add(second_location)
        self.locations[second_location].add(first_location)

    def display(self):
        """Return each location and its directly connected locations."""
        lines = []

        for location, neighbours in self.locations.items():
            connected = ", ".join(sorted(neighbours)) or "(no connections)"
            lines.append(f"{location}: {connected}")

        return "\n".join(lines)

    def find_route(self, start, destination):
        """Return the shortest route from start to destination using BFS.

        BFS works in these steps:
        1. Put the starting location in a queue and mark it visited.
        2. Remove the next location from the front of the queue.
        3. Add each unvisited neighbour to the queue and remember its parent.
        4. Repeat until the destination is found or the queue is empty.
        5. Follow the parent links backwards, then reverse that list.
        """

        if start not in self.locations or destination not in self.locations:
            return []

        queue = deque([start])
        parent = {start: None}

        while queue:
            current = queue.popleft()

            if current == destination:
                break

            for neighbour in sorted(self.locations[current]):
                if neighbour not in parent:
                    parent[neighbour] = current
                    queue.append(neighbour)

        if destination not in parent:
            return []

        route = []
        current = destination

        while current is not None:
```

```

        route.append(current)
        current = parent[current]

    return route[::-1]

def build_sample_hospital():
    """Build a sample hospital map for the demonstration and tests."""

    graph = HospitalGraph()

    for location in (
        "Main Entrance",
        "Emergency",
        "Radiology",
        "Surgery",
        "Pharmacy",
        "ICU",
    ):
        graph.add_node(location)

    graph.add_edge("Main Entrance", "Emergency")
    graph.add_edge("Main Entrance", "Radiology")
    graph.add_edge("Emergency", "Surgery")
    graph.add_edge("Radiology", "Pharmacy")
    graph.add_edge("Surgery", "ICU")
    graph.add_edge("Pharmacy", "ICU")

    return graph

class TestHospitalGraph(unittest.TestCase):

    def setUp(self):
        self.hospital = build_sample_hospital()

    def test_add_node_and_edge(self):
        self.hospital.add_node("Cafeteria")
        self.hospital.add_edge("Cafeteria", "Main Entrance")

        self.assertIn("Cafeteria", self.hospital.locations)
        self.assertIn(
            "Cafeteria",
            self.hospital.locations["Main Entrance"]
        )

    def test_display(self):
        self.assertIn(
            "Main Entrance: Emergency, Radiology",
            self.hospital.display()
        )

    def test_direct_route(self):
        self.assertEqual(
            self.hospital.find_route(
                "Main Entrance",
                "Emergency"
            ),
            ["Main Entrance", "Emergency"]
        )

    def test_shortest_multi_hop_route(self):
        self.assertEqual(
            self.hospital.find_route(
                "Main Entrance",
                "ICU"
            ),
            [
                "Main Entrance",
                "Emergency",
                "Surgery",
                "ICU",
            ]
        )

    def test_unreachable_route(self):
        self.hospital.add_node("Staff Housing")

        self.assertEqual(
            self.hospital.find_route(
                "Main Entrance",
                "Staff Housing"
            ),
            []
        )

```

```

def test_missing_location(self):
    self.assertEqual(
        self.hospital.find_route(
            "Main Entrance",
            "Laboratory"
        ),
        []
    )

if __name__ == "__main__":

    graph = build_sample_hospital()

    print("Hospital Graph - Adjacency List")
    print("-----")
    print(graph.display())

    print("\nHospital Routes using BFS")
    print("-----")

    test_routes = [
        ("Main Entrance", "ICU"),
        ("Emergency", "Pharmacy"),
        ("Radiology", "Surgery"),
    ]

    for start, destination in test_routes:

        route = graph.find_route(start, destination)

        if route:
            print(
                f"{start} -> {destination}: "
                f"' -> '.join(route)"
            )
        else:
            print(
                f"{start} -> {destination}: "
                "No route found"
            )

    print("\nRunning tests...")

    unittest.main(
        argv=["first-arg-is-ignored"],
        exit=False
    )

```

4. Output

Hospital Graph - Adjacency List

```

-----
Main Entrance: Emergency, Radiology
Emergency: Main Entrance, Surgery
Radiology: Main Entrance, Pharmacy
Surgery: Emergency, ICU
Pharmacy: ICU, Radiology
ICU: Pharmacy, Surgery

```

Hospital Routes using BFS

```

-----
Main Entrance -> ICU: Main Entrance -> Emergency -> Surgery -> ICU
Emergency -> Pharmacy: Emergency -> Main Entrance -> Radiology -> Pharmacy
Radiology -> Surgery: Radiology -> Main Entrance -> Emergency -> Surgery

```

Running tests...

.....

Ran 6 tests ...s

OK

5. Observation

The program represents hospital locations as nodes and the connections between them as undirected edges. An adjacency list is used to display the graph. Breadth-First Search (BFS) uses a queue to explore connected locations level by level and finds a shortest route in this unweighted graph. Three source-destination pairs were tested, and six unit tests passed

successfully.

TASK 3 | HEAP: EMERGENCY PATIENT PRIORITY

1. Question

2. Prompt

Implement Task 3 – Heap: Emergency Patient Priority Queue in Python. Use a min-heap so that lower priority numbers are served first. Support adding, viewing, serving/removing, and displaying patients. Handle tied priorities by preserving arrival order. Include tests and clear output demonstrating the required operations.

3. Code

```
"""task-3_Heap-based emergency patient priority queue."""

import heapq
import unittest

class EmergencyPatientQueue:
    """Serve emergency patients by severity, preserving arrival order for ties."""

    _PRIORITY_NAMES = {
        1: "Critical",
        2: "Serious",
        3: "Moderate",
        4: "Normal",
    }

    def __init__(self):
        self._patients = []
        self._arrival_number = 0

    def add_patient(self, name, priority):
        """Add a patient with a priority from 1 (highest) through 4 (lowest)."""
        if priority not in self._PRIORITY_NAMES:
            raise ValueError("priority must be between 1 and 4")

        patient = (priority, self._arrival_number, name)
        self._arrival_number += 1

        heapq.heappush(self._patients, patient)

    def peek_highest_priority(self):
        """Return the highest-priority patient without removing them."""
        if not self._patients:
            return None

        return self._format_patient(self._patients[0])

    def serve_patient(self):
        """Remove and return the highest-priority patient."""
        if not self._patients:
            return None

        return self._format_patient(heapq.heappop(self._patients))

    def display_waiting_patients(self):
        """Return patients in the order they will be served."""
        ordered_patients = sorted(self._patients)

        return [
            self._format_patient(patient)
            for patient in ordered_patients
        ]

    def _format_patient(self, patient):
        priority, _, name = patient

        return {
            "name": name,
            "priority": priority,
            "severity": self._PRIORITY_NAMES[priority],
        }

# -----
# Unit Tests
# -----

class TestEmergencyPatientQueue(unittest.TestCase):

    def setUp(self):
        self.queue = EmergencyPatientQueue()
```



```

def test_insertion_and_highest_priority(self):
    self.queue.add_patient("Alex", 3)
    self.queue.add_patient("Blair", 1)
    self.queue.add_patient("Casey", 2)

    self.assertEqual(
        self.queue.peek_highest_priority()["name"],
        "Blair"
    )

    self.assertEqual(
        len(self.queue.display_waiting_patients()),
        3
    )

def test_tied_priorities_preserve_arrival_order(self):
    self.queue.add_patient("Jordan", 2)
    self.queue.add_patient("Riley", 2)

    # Jordan arrived first, so Jordan is served first.
    self.assertEqual(
        self.queue.serve_patient()["name"],
        "Jordan"
    )

    self.assertEqual(
        self.queue.serve_patient()["name"],
        "Riley"
    )

def test_removal_order_and_empty_queue(self):
    self.queue.add_patient("Morgan", 4)
    self.queue.add_patient("Taylor", 1)
    self.queue.add_patient("Sam", 3)

    served_patients = [
        self.queue.serve_patient(),
        self.queue.serve_patient(),
        self.queue.serve_patient()
    ]

    expected = [
        {
            "name": "Taylor",
            "priority": 1,
            "severity": "Critical"
        },
        {
            "name": "Sam",
            "priority": 3,
            "severity": "Moderate"
        },
        {
            "name": "Morgan",
            "priority": 4,
            "severity": "Normal"
        }
    ]

    self.assertEqual(served_patients, expected)

    # Queue should be empty now.
    self.assertIsNone(self.queue.serve_patient())

def test_invalid_priority(self):
    with self.assertRaises(ValueError):
        self.queue.add_patient("Pat", 5)

# -----
# Demonstration
# -----

if __name__ == "__main__":

    queue = EmergencyPatientQueue()

    # Add sample patients.
    queue.add_patient("Ava", 3)
    queue.add_patient("Noah", 1)
    queue.add_patient("Mia", 1)
    queue.add_patient("Liam", 4)

    print("Emergency Patient Priority Queue")
    print("-----")

```

```

print("\nWaiting Patients:")
for patient in queue.display_waiting_patients():
    print(
        f"{patient['name']} - "
        f"Priority {patient['priority']} "
        f"({patient['severity']})"
    )

print("\nHighest Priority Patient:")
highest = queue.peak_highest_priority()

if highest:
    print(
        f"{highest['name']} - "
        f"Priority {highest['priority']} "
        f"({highest['severity']})"
    )

print("\nServing Patients:")
while queue.display_waiting_patients():
    patient = queue.serve_patient()

    print(
        f"Served: {patient['name']} - "
        f"Priority {patient['priority']} "
        f"({patient['severity']})"
    )

print("\nRunning tests...")

unittest.main(
    argv=["first-arg-is-ignored"],
    exit=False
)

```

4. Output

Emergency Patient Priority Queue

Waiting Patients:

Noah - Priority 1 (Critical)
 Mia - Priority 1 (Critical)
 Ava - Priority 3 (Moderate)
 Liam - Priority 4 (Normal)

Highest Priority Patient:

Noah - Priority 1 (Critical)

Serving Patients:

Served: Noah - Priority 1 (Critical)
 Served: Mia - Priority 1 (Critical)
 Served: Ava - Priority 3 (Moderate)
 Served: Liam - Priority 4 (Normal)

Running tests...

....

Ran 4 tests ... OK

5. Observation

The program uses a min-heap to keep the patient with the lowest priority number at the top of the queue. Priority 1 patients are served before lower-severity patients. When two patients have the same priority, their arrival number preserves the order in which they are served. Heap insertion and removal take $O(\log n)$, while viewing the highest-priority patient takes $O(1)$. The unit tests confirm insertion, priority ordering, tie handling, removal, empty queue behavior, and invalid priority handling.

TASK 4 | TRIE: MEDICAL WORD / DRUG SEARCH

1. Question

2. Prompt

Implement a Trie in Python for medical-word autocomplete. Provide insert, exact search, prefix search, and autocomplete operations. Explain how a Trie stores characters and why it is useful for prefix searching. Test multiple prefixes and display clear output.

3. Code

```
"""task-4_Trie-based autocomplete for medical words."""

import unittest

class TrieNode:
    """Store the next characters and whether this node ends a word."""

    def __init__(self):
        self.children = {}
        self.is_word = False

class Trie:
    """Store medical words for exact, prefix, and autocomplete searches."""

    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        """Insert a word into the Trie."""
        word = self._normalize_word(word)
        node = self.root

        for character in word:
            node = node.children.setdefault(character, TrieNode())

        node.is_word = True

    def search(self, word):
        """Return True when the complete word is stored."""
        node = self._find_node(word)
        return node is not None and node.is_word

    def prefix_search(self, prefix):
        """Return True when any stored word starts with the prefix."""
        return self._find_node(prefix) is not None

    def autocomplete(self, prefix, limit=None):
        """Return stored words beginning with prefix in alphabetical order."""
        if not isinstance(prefix, str):
            raise ValueError("prefix must be a string")

        if limit is not None and (not isinstance(limit, int) or limit < 0):
            raise ValueError("limit must be a non-negative integer or None")

        normalized_prefix = prefix.strip().lower()
        node = self._find_node(normalized_prefix)

        if node is None:
            return []

        matches = []
        self._collect_words(node, normalized_prefix, matches)

        if limit is not None:
            return matches[:limit]

        return matches

    @staticmethod
    def _normalize_word(word):
        if not isinstance(word, str) or not word.strip():
            raise ValueError("word must be a non-empty string")
        return word.strip().lower()

    def _find_node(self, text):
        if not isinstance(text, str):
            return None

        node = self.root
```

```

        for character in text.strip().lower():
            node = node.children.get(character)

            if node is None:
                return None

        return node

def _collect_words(self, node, prefix, matches):
    if node.is_word:
        matches.append(prefix)

    for character in sorted(node.children):
        self._collect_words(
            node.children[character],
            prefix + character,
            matches
        )

# -----
# Unit Tests
# -----

class TestTrie(unittest.TestCase):

    def setUp(self):
        self.trie = Trie()

        medical_words = (
            "cardiology",
            "cardiologist",
            "cardiac",
            "diabetes",
            "dialysis",
            "neurology",
            "neurotherapy",
        )

        for word in medical_words:
            self.trie.insert(word)

    def test_exact_search(self):
        self.assertTrue(self.trie.search("Cardiology"))
        self.assertFalse(self.trie.search("cardio"))

    def test_prefix_search(self):
        self.assertTrue(self.trie.prefix_search("card"))
        self.assertTrue(self.trie.prefix_search("dia"))
        self.assertTrue(self.trie.prefix_search("neuro"))
        self.assertFalse(self.trie.prefix_search("xyz"))

    def test_autocomplete(self):
        self.assertEqual(
            self.trie.autocomplete("card"),
            ["cardiac", "cardiologist", "cardiology"],
        )

        self.assertEqual(
            self.trie.autocomplete("dia"),
            ["diabetes", "dialysis"],
        )

        self.assertEqual(
            self.trie.autocomplete("neu"),
            ["neurology", "neurotherapy"],
        )

    def test_duplicate_insert_and_invalid_word(self):
        self.trie.insert("CARDIOLOGY")
        self.assertEqual(
            self.trie.autocomplete("cardiology"),
            ["cardiology"],
        )

        with self.assertRaises(ValueError):
            self.trie.insert(" ")

# -----
# Demonstration
# -----

if __name__ == "__main__":
    medical_words = Trie()

```

```

words = (
    "cardiology",
    "cardiologist",
    "cardiac",
    "diabetes",
    "dialysis",
    "neurology",
    "neurotherapy",
)

for word in words:
    medical_words.insert(word)

print("Medical Word Trie")
print("-----")

print("\nExact Search:")
print("Cardiology:", medical_words.search("Cardiology"))
print("cardio:", medical_words.search("cardio"))

print("\nPrefix Search:")
for prefix in ("card", "dia", "neuro"):
    print(f"{prefix}:", medical_words.prefix_search(prefix))

print("\nAutocomplete:")
for prefix in ("card", "dia", "neu"):
    print(f"{prefix}:", medical_words.autocomplete(prefix))

print("\nRunning tests...")

unittest.main(
    argv=["first-arg-is-ignored"],
    exit=False
)

```

4. Output

Medical Word Trie

Exact Search:

Cardiology: True

cardio: False

Prefix Search:

card: True

dia: True

neuro: True

Autocomplete:

card: ['cardiac', 'cardiologist', 'cardiology']

dia: ['diabetes', 'dialysis']

neu: ['neurology', 'neurotherapy']

Running tests...

....

Ran 4 tests ... OK

5. Observation

The Trie stores characters along shared paths, so words with common prefixes reuse the same character nodes. Exact search checks whether a complete word is stored, prefix search checks whether a character path exists, and autocomplete collects all stored words below the prefix node. Trie insertion and exact search take $O(L)$, where L is the word length. Prefix search takes $O(P)$, and autocomplete takes $O(P + K)$, where P is the prefix length and K represents the output size.

TASK 5 | AI DECISION CHALLENGE

1. Question

2. Prompt

Implement Task 5 – AI Decision Challenge in Python. For the following four hospital scenarios, choose the most suitable non-linear data structure from Tree, Graph, Heap, and Trie: hospital department hierarchy, hospital route finding between locations, emergency patient priority management, and medical word autocomplete. For each scenario, provide the selected data structure, why it is suitable, the main operation, and its time complexity using Big-O notation. Compare the choices with the AI assistant's suggestions and display the results clearly. Create a simple Python program that prints the decision table and comparison with the AI assistant.

3. Code

```
"""Task 5 - AI Decision Challenge."""

rows = [
    (
        "1. Hospital Department Hierarchy",
        "Tree",
        "Department traversal",
        "O(h)"
    ),
    (
        "2. Hospital Route Finding",
        "Graph",
        "BFS route finding",
        "O(V + E)"
    ),
    (
        "3. Emergency Patient Priority",
        "Heap",
        "Remove highest-priority patient",
        "O(log n)"
    ),
    (
        "4. Medical Word Autocomplete",
        "Trie",
        "Prefix search",
        "O(p + k)"
    ),
]

def display_decisions():
    print("AI Decision Challenge")
    print("=====")

    for scenario, structure, operation, complexity in rows:
        print(f"\n{scenario}")
        print(f"    Data Structure: {structure}")
        print(f"    Main Operation: {operation}")
        print(f"    Time Complexity: {complexity}")

    print("\nComparison with AI Assistant")
    print("=====")

    print("Our choices match the AI assistant's suggestions:")
    print("Tree -> Hospital Department Hierarchy")
    print("Graph -> Hospital Route Finding")
    print("Heap -> Emergency Patient Priority")
    print("Trie -> Medical Word Autocomplete")

if __name__ == "__main__":
    display_decisions()
```

4. Output

```
AI Decision Challenge
=====

1. Hospital Department Hierarchy
   Data Structure: Tree
   Main Operation: Department traversal
   Time Complexity: O(h)

2. Hospital Route Finding
   Data Structure: Graph
   Main Operation: BFS route finding
   Time Complexity: O(V + E)
```

3. Emergency Patient Priority
Data Structure: Heap
Main Operation: Remove highest-priority patient
Time Complexity: $O(\log n)$

4. Medical Word Autocomplete
Data Structure: Trie
Main Operation: Prefix search
Time Complexity: $O(p + k)$

Comparison with AI Assistant

=====

Our choices match the AI assistant's suggestions:

Tree -> Hospital Department Hierarchy

Graph -> Hospital Route Finding

Heap -> Emergency Patient Priority

Trie -> Medical Word Autocomplete

5. Observation

The selected data structures match the requirements of the four hospital scenarios. A Tree represents the hierarchical department structure, a Graph represents locations and routes, a Heap manages patients according to priority, and a Trie supports prefix-based medical word autocomplete. The main operations and their Big-O complexities are displayed in the program and the choices match the AI assistant's suggestions.